

EN İYİ UYGULAMALAR VE TEMİZ KOD

Bu bölümde Python'da profesyonel kod yazma prensiplerini, kod kalitesini artırma yöntemlerini ve yaygın kullanılan [en iyi uygulamaları](#) (best practice) inceleyeceğiz.

Python Stil Rehberi

PEP 8, Python kodu için resmi stil rehberidir. Kodun okunabilirliğini artırır.

Girinti

```
# DOĞRU
def hesapla(x, y):
    sonuc = x + y
    return sonuc

# YANLIŞ - Tab kullanma, 4 boşluk kullan!
def hesapla(x, y):
    sonuc = x + y
    return sonuc
```

Satır Uzunluğu

```
# DOĞRU - Maksimum 79 karakter
def uzun_fonksiyon_ismi(parametre1, parametre2,
                        parametre3, parametre4):
    return parametre1 + parametre2 + parametre3 + parametre4

# Uzun string'ler için
mesaj = (
    "Bu çok uzun bir mesajdır. "
    "Birden fazla satıra bölünmüştür. "
    "Okunabilirliği artırır."
)
```

İsmlendirme Kuralları

```
# Değişkenler ve fonksiyonlar: snake_case
kullanici_adi = "Ali"
toplam_fiyat = 100.50

def hesapla_toplam(fiyat, vergi):
    return fiyat * (1 + vergi)

# Sınıflar: PascalCase
class KullaniciYoneticisi:
    pass

class OgrenciKayit:
    pass

# Sabitler: UPPER_SNAKE_CASE
MAX_BAGLANTI = 100
API_ANAHTARI = "gizli-anahtar"
PI_DEGERI = 3.14159

# Private (özel) Üyeler: önünde _
class Hesap:
    def __init__(self):
        self._bakiye = 0 # Private
        self.__hesap_no = "12345" # Strongly private
```

Modülleri İç Aktarma Düzeni

İlk önce standart kütüphaneler, sonra üçüncü parti kütüphaneler, en son kendi modüllerimizi **iç aktarmalıyız** (**import**);

```
# 1. Standart kütüphane
import os
import sys
from datetime import datetime

# 2. Üçüncü parti kütüphaneler
import numpy as np
import pandas as pd
from fastapi import FastAPI

# 3. Kendi modülleriniz
from myapp.models import Kullanici
from myapp.utils import hesapla_toplam
```

Boşluklar

```
# DOĞRU
x = 5
y = x + 10
liste = [1, 2, 3]
sozluk = {"a": 1, "b": 2}

def fonksiyon(a, b, c):
    return a + b + c

# YANLIŞ
x=5
y=x+10
liste=[1,2,3]
sozluk={"a":1,"b":2}

def fonksiyon(a,b,c):
    return a+b+c
```

Dokümantasyon Metinleri

Fonksiyon, sınıf ve modüllerin dokümantasyonu için **docstring** kullanılmalıdır:

```
"""
Verilen sayıların toplamını hesaplar.

Args:
    sayilar (list[int]): Toplanacak sayılar listesi

Returns:
    int: Sayıların toplamı

Raises:
    ValueError: Liste boşsa

Examples:
    >>> toplam_hesapla([1, 2, 3])
    6
    >>> toplam_hesapla([10, 20])
    30
"""
if not sayilar:
```

```

        raise ValueError("Liste boş olamaz")
        return sum(sayilar)

class Kullanici:
    """
    Kullanıcı bilgilerini yöneten sınıf.

    Attributes:
        isim (str): Kullanıcının adı
        email (str): Kullanıcının email adresi
        yas (int): Kullanıcının yaşı
    """

    def __init__(self, isim: str, email: str, yas: int):
        self.isim = isim
        self.email = email
        self.yas = yas

```

Tür İpuçlarını Kullanın

Kod okunabilirliği ve IDE desteği için **tür ipuçlarını** (**type hint**) kullanın:

```

def filtrele(
    sayilar: List[int],
    kosul: Callable[[int], bool]
) -> List[int]:
    """Verilen koşulu sağlayan sayıları filtreler."""
    return [s for s in sayilar if kosul(s)]

def kullanici_bul(
    kullanici_id: int
) -> Optional[Dict[str, Union[str, int]]]:
    """Kullanıcı bulunursa dict, bulunamazsa None döner."""
    if kullanici_id > 0:
        return {"id": kullanici_id, "isim": "Ali"}
    return None

```

SOLID İlkeleri

S-Single Responsibility (Tek Sorumluluk)

Her sınıf veya fonksiyon tek bir işten sorumlu olmalı:

```

# YANLIŞ - Birden fazla sorumluluk
class Kullanici:
    def __init__(self, isim, email):
        self.isim = isim
        self.email = email

    def kaydet(self):
        # Veritabanına kaydet
        pass

    def email_gonder(self):
        # Email gönder
        pass

    def rapor_olustur(self):
        # Rapor oluştur
        pass

# DOĞRU - Her sınıf tek bir işten sorumlu

```

```
class Kullanici:
    def __init__(self, isim, email):
        self.isim = isim
        self.email = email

class KullaniciRepository:
    def kaydet(self, kullanici: Kullanici):
        # Veritabanına kaydet
        pass

class EmailServisi:
    def gonder(self, alici: str, mesaj: str):
        # Email gönder
        pass

class RaporServisi:
    def olustur(self, kullanici: Kullanici):
        # Rapor oluşturun
        pass
```

O-Open/Closed (Açık/Kapalı)

Genişletmeye açık, değişikliğe kapalı olmalı:

```
# YANLIŞ - Yeni şekil eklemek için sınıfı değiştirmek gerekir
class AlanHesaplama:
    def hesapla(self, sekil):
        if sekil.tip == "daire":
            return 3.14 * sekil.yaricap ** 2
        elif sekil.tip == "kare":
            return sekil.kenar ** 2

# DOĞRU - Abstract base class kullanımı
from abc import ABC, abstractmethod

class Sekil(ABC):
    @abstractmethod
    def alan_hesapla(self) -> float:
        pass

class Daire(Sekil):
    def __init__(self, yaricap: float):
        self.yaricap = yaricap

    def alan_hesapla(self) -> float:
        return 3.14 * self.yaricap ** 2

class Kare(Sekil):
    def __init__(self, kenar: float):
        self.kenar = kenar

    def alan_hesapla(self) -> float:
        return self.kenar ** 2

# Yeni şekil eklemek kolay:
class Ucgen(Sekil):
    def __init__(self, taban: float, yukseklik: float):
        self.taban = taban
        self.yukseklik = yukseklik

    def alan_hesapla(self) -> float:
        return (self.taban * self.yukseklik) / 2
```

L-Liskov Substitution (Liskov İkame)

Alt sınıf, üst sınıfın yerine kullanılabilirmeli:

```
# DOĞRU
# class Kus:
#     def ucabilir_mi(self) -> bool:
#         return True

# class Serce(Kus):
#     pass # Uçabilir

# class Penguen(Kus):
#     def ucabilir_mi(self) -> bool:
#         return False # Uçamaz ama davranış tutarlı

# Kullanım
# def kus_ucur(kus: Kus):
#     if kus.ucabilir_mi():
#         print(f"{kus.__class__.__name__} uçuyor")
#     else:
#         print(f"{kus.__class__.__name__} uçamıyor")
```

I-Interface Segregation (Arayüz Ayrımı)

Sınıflar kullanmadıkları yöntemleri kodlamaya zorlanmamalı:

```
# YANLIŞ
class Isci(ABC):
    @abstractmethod
    def calis(self): pass

    @abstractmethod
    def yemek_ye(self): pass

    @abstractmethod
    def uyku_mo_ver(self): pass

class Robot(Isci): # Robot yemek yemez ve uyumaz!
    def calis(self): pass
    def yemek_ye(self): pass # Gereksiz
    def uyku_mo_ver(self): pass # Gereksiz

# DOĞRU - Küçük interface'ler
class Calisabilir(ABC):
    @abstractmethod
    def calis(self): pass

class Beslenebilir(ABC):
    @abstractmethod
    def yemek_ye(self): pass

class Dinlenebilir(ABC):
    @abstractmethod
    def uyku_mo_ver(self): pass

class Insan(Calisabilir, Beslenebilir, Dinlenebilir):
    def calis(self): pass
    def yemek_ye(self): pass
    def uyku_mo_ver(self): pass

class Robot(Calisabilir):
```

```
def calis(self): pass
```

D-Dependency Inversion

Bağımlılık tersine çevirme olarak bilinen bu prensip üst seviye modüller alt seviye modüllere bağımlı olmamasını ister:

```
# YANLIŞ - Doğrudan bağımlılık
class MySQLVeritabani:
    def kaydet(self, veri):
        print("MySQL'e kaydedildi")

class KullaniciServisi:
    def __init__(self):
        self.db = MySQLVeritabani() # Sıkı bağlantı

    def kullanıcı_ekle(self, kullanıcı):
        self.db.kaydet(kullanıcı)

# DOĞRU - Abstraction kullanımı
class Veritabani(ABC):
    @abstractmethod
    def kaydet(self, veri): pass

class MySQLVeritabani(Veritabani):
    def kaydet(self, veri):
        print("MySQL'e kaydedildi")

class PostgreSQLVeritabani(Veritabani):
    def kaydet(self, veri):
        print("PostgreSQL'e kaydedildi")

class KullaniciServisi:
    def __init__(self, veritabani: Veritabani):
        self.db = veritabani # Dependency injection

    def kullanıcı_ekle(self, kullanıcı):
        self.db.kaydet(kullanıcı)

# Kullanım
mysql_db = MySQLVeritabani()
servis = KullaniciServisi(mysql_db)

postgres_db = PostgreSQLVeritabani()
servis2 = KullaniciServisi(postgres_db)
```

Tekrarlardan Kaçınma İlkesi

Bu ilki şu şekilde ifade edilir: "Her bilgi parçasının bir sistem içinde tek, açık ve güvenilir bir temsili olmalıdır". Yani aynı görevi gören kod tekrarından kaçının (**Don't Repeat Yourself-DRY**):

```
# YANLIŞ - Kod tekrarı
def kullanıcı_ekle(isim, email):
    if not isim:
        print("Hata: İsim boş olamaz")
        return False
    if not email:
        print("Hata: Email boş olamaz")
        return False
    if "@" not in email:
        print("Hata: Geçersiz email")
        return False
```

```

# Kullanıcı ekle
return True

def urun_ekle(isim, fiyat):
    if not isim:
        print("Hata: İsim boş olamaz")
        return False
    if fiyat < 0:
        print("Hata: Fiyat negatif olamaz")
        return False
    # Ürün ekle
    return True

# DOĞRU - Validation fonksiyonu
def dogrula(deger, hata_mesaji):
    if not deger:
        print(f"Hata: {hata_mesaji}")
        return False
    return True

def kullanıcı_ekle(isim, email):
    if not dogrula(isim, "İsim boş olamaz"):
        return False
    if not dogrula(email, "Email boş olamaz"):
        return False
    if "@" not in email:
        print("Hata: Geçersiz email")
        return False
    # Kullanıcı ekle
    return True

```

Aptala Anlatır Gibi Basit İlkesi

Bu tasarım ve geliştirme ilkesi, sistemlerin, ürünlerin ve süreçlerin gereksiz karmaşıklıktan kaçınılarak basit tutulması gerektiğini savunur. Basit tutun, gereksiz karmaşıklıktan kaçının (**Keep It Simple, Stupid-KISS**):

```

# YANLIŞ - Gereksiz karmaşık
def cift_mi(sayi):
    if sayi % 2 == 0:
        return True
    else:
        return False

# DOĞRU - Basit
def cift_mi(sayi):
    return sayi % 2 == 0

```

Code Smell'ler ve Çözümleri

Long Method (Uzun Fonksiyon)

```

# YANLIŞ
def siparis_isle(siparis):
    # 100+ satır kod
    # Stok kontrolü
    # Fiyat hesaplama
    # Kargo hesaplama
    # Email gönderme
    # Veritabanı kayıt
    pass

```

```
# DOĞRU
def siparis_isle(siparis):
    stok_kontrol(siparis)
    fiyat = fiyat_hesapla(siparis)
    kargo = kargo_hesapla(siparis)
    email_gonder(siparis)
    veritabanına_kaydet(siparis)
```

Magic Numbers (Sihirli Sayılar)

```
# YANLIŞ
def indirim_hesapla(fiyat):
    if fiyat > 1000:
        return fiyat * 0.15
    elif fiyat > 500:
        return fiyat * 0.10
    return fiyat * 0.05

# DOĞRU
YUKSEK_INDIRIM_SINIRI = 1000
ORTA_INDIRIM_SINIRI = 500
YUKSEK_INDIRIM_ORANI = 0.15
ORTA_INDIRIM_ORANI = 0.10
DUSUK_INDIRIM_ORANI = 0.05

def indirim_hesapla(fiyat):
    if fiyat > YUKSEK_INDIRIM_SINIRI:
        return fiyat * YUKSEK_INDIRIM_ORANI
    elif fiyat > ORTA_INDIRIM_SINIRI:
        return fiyat * ORTA_INDIRIM_ORANI
    return fiyat * DUSUK_INDIRIM_ORANI
```

Kod Kalitesi Araçları

Kod biçimlendirme aracı (Black):

```
pip install black
black dosya.py
```

Kod Analiz aracı (Pylint):

```
pip install pylint
pylint dosya.py
```

Tip kontrolü aracı (mypy):

```
pip install mypy
mypy dosya.py
```

Hızlı [SQL biçimlendirici](#) (linter) aracı (ruff):

```
pip install ruff
ruff check dosya.py
```

İz Kayıtları Oluşturma

[İz kaydı](#) (log) oluşturmak için `print()` yerine `logging` kullanın:

```
# Logging ayarları
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
)
```

```

logger = logging.getLogger(__name__)

def kullanıcı_ekle(isim, email):
    logger.info(f"Yeni kullanıcı ekleniyor: {isim}")
    try:
        # Kullanıcı ekle
        logger.debug(f"Email doğrulanıyor: {email}")
        # ...
        logger.info("Kullanıcı başarıyla eklendi")
        return True
    except Exception as e:
        logger.error(f"Kullanıcı eklenirken hata: {str(e)}")
        return False

```

Konfigürasyon Yönetimi

```

# config.py
import os
from dotenv import load_dotenv

load_dotenv()

class Config:
    DEBUG = os.getenv("DEBUG", "False") == "True"
    DATABASE_URL = os.getenv("DATABASE_URL")
    SECRET_KEY = os.getenv("SECRET_KEY")
    MAX_BAGLANTI = int(os.getenv("MAX_BAGLANTI", "100"))

```

Konfigürasyona yönelik tüm bilgiler `env` dosyasında olmalıdır;

```

# .env dosyası
DEBUG=True
DATABASE_URL=postgresql://user:pass@localhost/db
SECRET_KEY=gizli-anahtar
DB_PASSWORD=Password123
MAX_BAGLANTI=50

```

Aşağıda güvenlikle ilgili doğru ve yanlış kullanım gösterilmiştir;

```

# YANLIŞ Kullanım:
password = "12345" # Kodda hardcoded
sql = f"SELECT * FROM users WHERE id = {user_id}" # SQL injection

# GÜVENLİ Kullanım
import os
password = os.getenv('DB_PASSWORD')

# Parametrelili query
cursor.execute("SELECT * FROM users WHERE id = ?", (user_id,))

```

Bağlam Yöneticisi

Herhangi bir programlama dilinde, dosya işlemleri veya veri tabanı bağlantıları gibi kaynakların kullanımı çok yaygındır, ancak sınırlıdır. Bu nedenle, asıl sorun, bu kaynakların kullanımdan sonra serbest bırakılmasını sağlamaktır. Serbest bırakılmazlarsa, kaynak sızıntısına yol açar ve sistemin yavaşlamasına veya çökmesine neden olabilir.

Python'ın **bağlam yöneticileri** (**context manager**), kaynakları otomatik olarak kurmak ve temizlemek için düzgün bir yol sağlar ve hatalar oluşsa bile düzgün bir şekilde yönetilmelerini sağlar.

Bağlam Yöneticilerine Neden İhtiyacımız Var?

- **Otomatik temizleme:** Dosyalar veya bağlantılar gibi kaynakları manuel çaba gerektirmeden serbest bırakır.
- **Kaynak sızıntısını önleme:** Hatalar oluşsa bile düzgün serbest bırakmayı sağlar.
- **Daha temiz kod:** Ayrıntılı `try-finally` bloklarını ile değiştirir.
- **İstisna güvenli:** İstisnalarda bile her zaman temizleme işlemi gerçekleştirir.
- **Özel kontrol:** `__enter__` ve `__exit__` kullanarak herhangi bir kaynağı yönetmenizi sağlar.

```
# YANLIŞ
dosya = open("veri.txt", "r")
icerik = dosya.read()
dosya.close() # Hata olursa kapatılmayabilir!

# DOĞRU
with open("veri.txt", "r") as dosya:
    icerik = dosya.read()
    # Otomatik kapatılır

# Kendi context manager'iniz
from contextlib import contextmanager

@contextmanager
def veritabani_baglantisi():
    baglanti = veritabani_baglan()
    try:
        yield baglanti
    finally:
        baglanti.kapat()

with veritabani_baglantisi() as db:
    db.sorgu_calistir("SELECT * FROM users")
```

Liste veya Sözlük Kavrama

Kavramalar (**comprehension**) gereksiz uzun yazılmamalıdır.

```
# YANLIŞ - Gereksiz uzun
kareler = []
for i in range(10):
    kareler.append(i ** 2)

# DOĞRU - Kısa ve öz
kareler = [i ** 2 for i in range(10)]

# Dict comprehension
sozluk = {i: i ** 2 for i in range(10)}

# Koşullu
cift_kareler = [i ** 2 for i in range(10) if i % 2 == 0]
```

En İyi Uygulamalar

- PEP 8'e uyun - Kod stili tutarlı olsun
- Type hints kullanın - Kod okunabilirliği artar
- Docstring yazın - Her fonksiyon dokümanite edilmeli
- DRY prensibine uyun - Kod tekrardan kaçın
- SOLID prensiplerini uygulayın - Sürdürülebilir kod yazın
- Test yazın - Her fonksiyon test edilmeli
- Logging kullanın - `print()` yerine logging
- Environment variables kullanın - Hassas bilgileri kodda saklamayın

- Virtual environment kullanın - Proje bağımlılıklarını izole edin
- Git kullanın - Versiyon kontrolü yapın
- Code review yapın - Kodunuzu başkalarına inceletin
- Linter ve formatter kullanın - Kod kalitesini artırın
- CI/CD pipeline kurun - Otomatik test ve deployment
- Exception handling yapın - Hataları yakalayın
- Security best practices uygulayın - Güvenlik açıklarından kaçının